Informe Técnico de Auditoría de Código Analítico

Destinatario: Equipo de Ingeniería y BI

Rol: Arquitecto de Datos Senior

Asunto: Análisis técnico y optimización de consulta analítica de clientes VIP

---

1. Propósito de Negocio

El objetivo principal de esta consulta es **identificar y segmentar a los clientes de alto valor (VIP) por categoría de producto** a partir del ejercicio fiscal 2026 (`OrderDate >= '2026-01-01'`).

Específicamente, la consulta permite:

• **Focalizar estrategias comerciales:** Detectar qué clientes superan el umbral de facturación de **$15,000** en una categoría determinada.

• **Evaluar la recurrencia vs. volumen:** Al cruzar las ventas totales con el conteo de pedidos únicos (`TotalPedidos`), el negocio puede distinguir entre clientes que compran por volumen esporádico (pocas órdenes grandes) y clientes con alta recurrencia operativa (muchas órdenes medianas/pequeñas).

• **Cross-Selling / Upselling:** Permite a los Key Account Managers (KAM) enfocar esfuerzos en categorías complementarias donde estos clientes VIP aún no han alcanzado volúmenes significativos.

---

2. Tablas, Dimensiones y Medidas Involucradas

El diseño responde a un modelo transaccional normalizado (3NF) o copo de nieve (*Snowflake Schema*):

Tablas

• **Hechos (Transaccionales):**

• `Sales.Orders` (o): Cabecera de pedido (contiene fecha y cliente).

• `Sales.OrderDetails` (od): Detalle de línea de pedido (contiene granularidad a nivel producto, precio y cantidad).

• **Dimensiones:**

• `Sales.Customers` (cust): Dimensión de Cliente.

• `Sales.Products` (p): Dimensión de Producto.

• `Sales.Categories` (c): Sub-dimensión de Categoría normalizada desde Productos.

Dimensiones (Atributos de Agrupación)

• `c.CategoryName`: Categoría del producto adquirido.

• `cust.CustomerName`: Nombre del cliente.

Medidas (Métricas Calculadas)

• **TotalVentas:** `SUM(od.Quantity * od.UnitPrice)` — Venta bruta monetaria agregada.

• **TotalPedidos:** `COUNT(DISTINCT o.OrderID)` — Conteo escalar de órdenes distintas para evitar duplicidad generada por el `JOIN` a nivel de línea.

---

3. Lógica de Procesamiento y Recomendaciones de Optimización

Lógica de Ejecución (Query Engine Pipeline)

1. **Filtrado inicial (WHERE):** Evalúa `o.OrderDate >= '2026-01-01'`.

2. **Operaciones de Join:** Realiza 4 operaciones de combinación (`INNER JOIN`) entre cabecera, detalle y dimensiones.

3. **Agrupación y Agregación (Stream/Hash Aggregate):** Agrupa por `CategoryName` y `CustomerName`, calculando la suma de ventas y resolviendo el costo computacional del `DISTINCT` para los pedidos.

4. **Filtro Post-Agregación (HAVING):** Descarta aquellos grupos cuyo `TotalVentas <= 15000`.

5. **Ordenamiento (Sort):** Ordena de mayor a menor según `TotalVentas`.

Diagnóstico y Recomendaciones de Optimización

A. Estrategia de Indexación (SQL Server Engine)

Para evitar escaneos de tabla completos (*Table/Clustered Index Scans*) y costosos operadores de ordenamiento:

1. **Tabla de Cabeceras (`Sales.Orders`):**

```sql
CREATE NONCLUSTERED INDEX IX_Orders_OrderDate_Includes
ON Sales.Orders (OrderDate)
INCLUDE (OrderID, CustomerID);
```

Impacto: Permite un *Index Seek* eficiente por rango de fecha cubriendo las claves de relación.

2. **Tabla de Detalle (`Sales.OrderDetails`):**

```sql
CREATE NONCLUSTERED INDEX IX_OrderDetails_OrderID_Includes
ON Sales.OrderDetails (OrderID)
INCLUDE (ProductID, Quantity, UnitPrice);
```

Impacto: Evita *Key Lookups* para traer el precio y la cantidad en el cálculo de `TotalVentas`.

B. Costo del `COUNT(DISTINCT)`

El `COUNT(DISTINCT o.OrderID)` genera un operador de re-particionado o agregación en dos fases en el plan de ejecución, lo cual suele volcar datos a `tempdb` (*Spills*) en tablas volumétricas.

• *Alternativa de modelado:* Si esta consulta corre en un entorno analítico (DWH/Data Mart), se recomienda migrar a un **Columnstore Index (CCI)** en las tablas de hechos para aprovechar el modo por lotes (*Batch Mode Execution*), el cual vectoriza el cómputo del `COUNT(DISTINCT)`.

C. Integridad Financiera (Regla de Negocio)

• La fórmula actual `od.Quantity * od.UnitPrice` omite posibles descuentos (`Discount`). Si la tabla posee descuentos, el cálculo real de negocio debería ser:

```sql
```

SUM(od.Quantity * od.UnitPrice * (1 - COALESCE(od.Discount, 0)))
```

---

## Enfoque Alternativo para BI Moderno (Modelado Tabular / DAX)

Si este reporte migra a una capa semántica (Power BI / SSAS Tabular) con un modelo en estrella (*Star Schema* donde `OrderDetails` y `Orders` se colapsan o se relacionan 1:*), las medidas equivalentes optimizadas mediante el motor VertiPaq serían:

```dax
// Medida: Total Ventas
Total Ventas =
SUMX(
SalesOrderDetails,
SalesOrderDetails[Quantity] * SalesOrderDetails[UnitPrice]
)
// Medida: Total Pedidos
Total Pedidos =
DISTINCTCOUNT(SalesOrders[OrderID])
// Medida de Filtrado VIP (Evaluada en el contexto de filtro de Categoría y Cliente)
EsVentaVIP =
IF([Total Ventas] > 15000, 1, 0)
```

Ventaja: El motor VertiPaq calcula `DISTINCTCOUNT` de forma columnar y en memoria a un costo algorítmico significativamente menor que el motor relacional tradicional.

```